Manual de usuario: TacticalGrid

Integrantes: Cristian David Gonzalez - Sebastian Garcia Hincapie

Docente

Jairo Iván Vélez Bedoya

Sistemas Inteligentes I, Proyecto 1. Universidad de Caldas.

2026-2

Tabla de contenido

Manual de usuario: TacticalGrid..... 1
Requisitos..... 3
Instalación..... 3
Abrir la aplicación..... 3
La ventana..... 4
Cargar un escenario..... 5
Modo búsqueda..... 5
Modo adversarial..... 6
Modo partida..... 6
Verificar un resultado..... 7
Copiar y limpiar resultados..... 7
Formato del escenario (JSON)..... 7
Escenarios incluidos..... 8
Pruebas y experimentos..... 8
Problemas comunes..... 9

Manual de usuario: TacticalGrid

TacticalGrid es un juego táctico por turnos sobre una cuadrícula en el que dos bandos, A y B, compiten por llevar un recurso a su base. La aplicación permite cargar un escenario en formato JSON resolver navegaciones con BFS, DFS, UCS, A* y Beam Search, decidir jugadas con Minimax y alfa-beta, jugar partidas completas entre dos IA y verificar cada resultado. Este manual explica cómo instalarla y usarla.

Requisitos

- Python 3.11 o superior. En Windows, al instalar Python debe quedar marcada la opción "tcl/tk and IDLE", que incluye Tkinter.
- No se necesitan bibliotecas externas para usar la aplicación. Las pruebas y los experimentos usan pytest, ruff, pandas y matplotlib que se instalan con el grupo de desarrollo.

Instalación

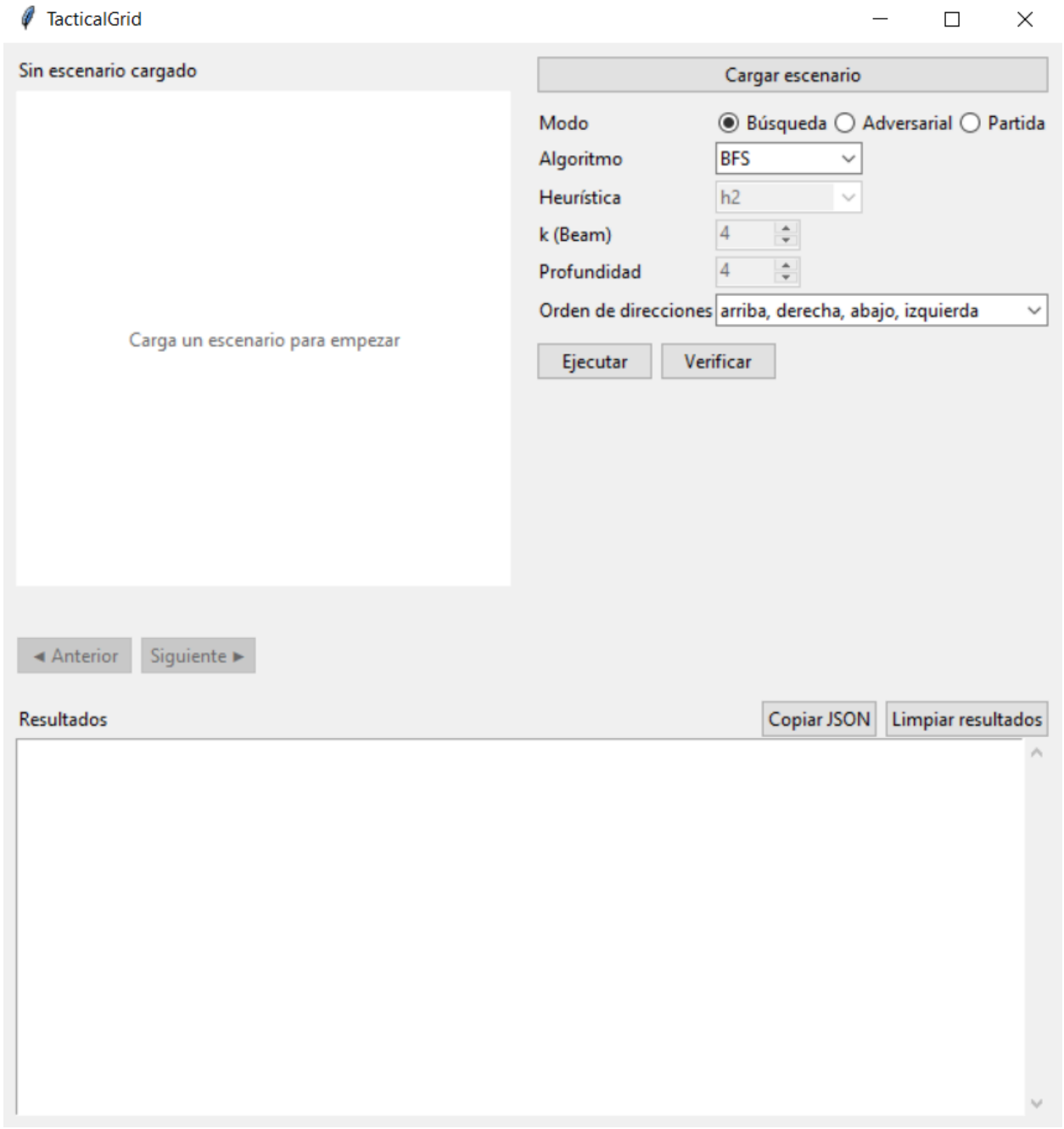
Desde la carpeta raíz del proyecto (donde está pyproject.toml), en una terminal de Windows:

1. Crear el entorno virtual: python -m venv .venv
2. Instalar el proyecto con las herramientas de desarrollo: “pip install -e ".[dev]"

Abrir la aplicación

python -m tacticalgrid
Se abre la ventana de TacticalGrid. Si se prefiere se puede activar antes el entorno con .venv\Scripts\activate y luego ejecutar python -m tacticalgrid.

La ventana



- **Mapa (izquierda):** la cuadrícula con un color por tipo de terreno, las celdas no transitables (muros) en gris oscuro, las bases con un borde del color de su bando, el recurso como un rombo dorado, las unidades como círculos azules (bando A) y rojos

(bando B) con su identificador. La unidad que lleva el recurso tiene un borde dorado.

- **Leyenda:** Debajo del mapa, cada tipo de terreno con su costo.
- **Línea de estado:** El turno actual, el portador del recurso y los turnos jugados del estado que se muestra.
- **Panel de control (derecha):** Botón Cargar escenario, modo, algoritmo, heurística, k (Beam), profundidad, orden de direcciones y los botones Ejecutar y Verificar.
- **Resultados (abajo):** Un resumen de cada ejecución. Las ejecuciones se acumulan una debajo de otra para poder compararlas. Los botones Copiar JSON y Limpiar resultados están junto a este panel.

Cargar un escenario

1. Pulsar **Cargar escenario** y elegir un archivo .json, por ejemplo scenarios/dev/enunciado_4x4.json.
2. Si el archivo es válido, se dibuja el mapa, el modo queda preseleccionado según el campo prueba.modo del JSON y la profundidad según juego.profundidad_maxima_minimax (4 si no viene).
3. Si el archivo tiene errores, aparece un mensaje con la lista completa de errores encontrados y la aplicación conserva el escenario que estaba cargado antes. La carga es completa o no se aplica.

Modo búsqueda

Resuelve la navegación de una unidad que pide la prueba del JSON: llegar a una celda objetivo, o con la misión llevar_recurso, recoger el recurso y llevarlo a la base de su bando. En navegación solo cuenta el terreno; las demás unidades no bloquean.

1. Elegir el modo **Búsqueda**.
2. Elegir el algoritmo: BFS, DFS, UCS, A* o Beam.
3. Para A* y Beam, elegir la heurística: h1 (Manhattan) o h2 (pasos rodeando obstáculos). Para Beam, elegir k.
4. Opcionalmente cambiar el orden de direcciones (afecta sobre todo a DFS).

5. Pulsar **Ejecutar**. El camino encontrado se dibuja sobre el mapa y en Resultados aparecen el camino, el costo, los movimientos, los estados generados y expandidos, la frontera máxima y el tiempo. Si no existe camino se indica.

El JSON debe tener una prueba en modo busqueda; si no, la aplicación muestra un mensaje explicando el problema.

Estados expandidos: Además del camino, el mapa marca con un punto pequeño cada celda que el algoritmo expandió. Un punto azul (esquina superior izquierda) indica que la celda se expandió sin el recurso un punto naranja (esquina superior derecha) indica que se expandió llevando el recurso. Una celda puede tener los dos puntos porque el estado de navegación incluye si la unidad lleva el recurso y para el algoritmo, son dos estados distintos. Debajo del mapa aparece la leyenda de ambos puntos y en Resultados la línea "Estados expandidos: N · celdas distintas: M". Copiar JSON incluye la lista completa, en orden, en la clave orden_expansion.

Comparar los puntos de varios algoritmos sobre el mismo mapa muestra cuánto exploró cada uno. Por ejemplo, en caso6_mision_grande.json BFS expande 148 estados y cubre prácticamente todo el mapa, mientras que A* con h2 expande 21, casi solo el camino. Las marcas se borran al cargar otro escenario, cambiar de modo o limpiar resultados.

Modo adversarial

Evalúa el estado del JSON y recomienda una jugada para el bando que tiene el turno. No juega la partida: responde cuál es la mejor jugada en ese momento.

1. Elegir el modo **Adversarial** y el algoritmo: Minimax o Alfa-beta.
2. Ajustar la profundidad (cuántas jugadas hacia adelante se analizan) y si se desea, el orden de direcciones.
3. Pulsar **Ejecutar**. La unidad recomendada y su casilla de destino se resaltan en el mapa. En Resultados aparecen la acción seleccionada, el valor, la profundidad, los nodos generados, evaluados y podados, y el tiempo.

Para comparar Minimax y alfa-beta, ejecutar ambos con la misma profundidad y el mismo orden: deben dar la misma acción y el mismo valor, y alfa-beta debe generar menos nodos.

Jugadas consideradas: Debajo de las métricas, Resultados lista cada jugada legal del bando que juega, en el orden en que se generó, con el valor que obtuvo. La jugada elegida se marca con ► y "← elegida"; es la de mayor valor y, si hay empate gana la que se generó primero.

Con Minimax todos los valores son exactos. Con Alfa-beta algunos aparecen como "≤ v (cota: rama podada)": esa jugada vale v o menos. Alfa-beta dejó de explorarla en cuanto encontró una respuesta del rival que la dejaba igual o peor que la mejor jugada ya asegurada no necesitó su valor exacto porque ya sabía que no supera a la elegida.

Modo partida

Juega una partida completa entre dos IA desde el estado del JSON. En cada turno decide el bando que tiene el turno con el algoritmo elegido hasta que un bando gana o se alcanza el límite de turnos (empate).

1. Elegir el modo **Partida**, el algoritmo y la profundidad.
2. Pulsar **Ejecutar**. En Resultados aparecen el ganador (o el empate por límite), el número de turnos y la lista de jugadas.
3. Recorrer la partida con ◂ **Anterior** y **Siguiente** ▸. El mapa muestra el estado de cada turno y la etiqueta indica el turno y la jugada realizada.

Con profundidades altas y mapas grandes la partida puede tardar unos segundos; mientras calcula, el cursor muestra el estado de espera.

Verificar un resultado

Después de cualquier ejecución, pulsar **Verificar**. La aplicación comprueba el último resultado sin depender del algoritmo que lo produjo:

- En búsqueda: que cada movimiento sea válido, que ninguna unidad atraviese obstáculos, que el costo reportado coincida con la suma de los terrenos y que el estado final cumpla el objetivo.
- En adversarial: que la jugada recomendada sea legal en el estado analizado.
- En partida: que cada jugada fuera legal, que no se generaran estados inválidos y que el ganador declarado sea correcto.

El resultado aparece en el panel como "Verificado ✔" o como la lista de errores encontrados.

Copiar y limpiar resultados

- **Copiar JSON** copia al portapapeles los datos completos de la última ejecución en formato JSON para pegarlos en otro documento o compararlos.
- **Limpiar resultados** borra el panel de resultados.

Formato del escenario (JSON)

Cada escenario es un archivo JSON con estos campos:

- **version:** versión del formato, por ejemplo "1.0".
- **mapa:** número de filas y columnas.
- **tipos_terreno:** catálogo de terrenos; cada uno indica si es transitable y, si lo es, su costo positivo.
- **terreno:** matriz de nombres de terreno con las dimensiones del mapa.
- **bases:** posición (fila, columna) de la base de cada bando, A y B.
- **recurso:** posición del recurso.
- **unidades:** lista de unidades con id, bando, tipo y posición.
- **turno:** bando que juega primero.
- **juego:** portador_recurso (id de la unidad que lleva el recurso, o null) y profundidad_maxima_minimax.
- **prueba (opcional):** la operación a realizar. En modo busqueda: unidad_inicio y objetivo. Los otros modos son adversarial y partida.

Campos propios del equipo, ambos opcionales: **prueba.mision** con el valor "llevar_recurso" pide recoger el recurso y llevarlo a la base en lugar de llegar a un objetivo; **juego.limite_turnos** fija el máximo de turnos de una partida (200 si no viene). Un escenario sin estos campos funciona con los valores por defecto.

Al cargar, la aplicación rechaza el archivo si las dimensiones no coinciden con la matriz, si un terreno no existe en el catálogo, si un terreno transitable no tiene costo positivo, si una base, el recurso o una unidad está fuera del mapa, si una unidad está sobre una celda no transitable, si hay

ids repetidos, si un bando o el turno no es A o B, o si el portador no es una unidad existente.

Escenarios incluidos

- scenarios/dev/enunciado_4x4.json: el escenario de ejemplo del enunciado (búsqueda de A1 hasta la celda (3,2)).
- scenarios/casos/caso3_muro.json: un muro entre la unidad y el objetivo; muestra la diferencia entre las heurísticas h1 y h2.
- scenarios/casos/caso6_mision.json: misión llevar_recurso en el mapa del enunciado; sirve también para comparar Beam Search con distintos k.
- scenarios/casos/caso6_mision_grande.json: misión en un mapa de 8 × 10 con el recurso en un callejón.
- scenarios/casos/caso7_8_10_adversarial.json: el mapa del enunciado en modo adversarial, para Minimax, alfa-beta y el efecto de la profundidad.
- scenarios/casos/caso9_orden.json: el mapa grande en modo adversarial, para el efecto del orden de direcciones en la poda.
- scenarios/invalidos/: un archivo con un error por cada regla de validación.

Pruebas y experimentos

- Ejecutar las pruebas automáticas: .venv\Scripts\python -m pytest
- Revisar el estilo del código: .venv\Scripts\ruff check .
- Reproducir los experimentos del informe: .venv\Scripts\python experiments\ejecutar_experimentos.py. Las tablas (CSV), las gráficas (PNG) y el archivo resumen.txt quedan en experiments/resultados/. La ejecución completa tarda algunos minutos.

Problemas comunes

- **"No module named tkinter":** Python se instaló sin Tcl/Tk. Abrir el instalador de Python, elegir Modify y marcar "tcl/tk and IDLE".
- **"No module named tacticalgrid":** la terminal no está en la carpeta raíz del proyecto o

falta la instalación. Ejecutar .venv\Scripts\pip install -e ".[dev]" desde la raíz.

- **La ventana no responde durante unos segundos:** es normal en partidas con profundidad alta en mapas grandes; la aplicación está calculando.
- **Un escenario no carga:** leer el mensaje de error, que indica el campo y la regla que no se cumple, y corregir el JSON.